



多模态大模型项目期末报告

题目： 基于强化学习决策的文档理解与检索
多模态问答系统

小组成员 刘天翔 杨毅涵 郭翊涛

学 号 23320104 23330154 22308043

院 系 计算机学院

专 业 计算机科学与技术

2026 年 6 月 19 日

摘 要

面向 PDF、扫描文档和图表型页面的问答任务，传统纯文本 RAG 容易丢失版面结构、图表数值和视觉区域位置，导致证据定位与回答生成不稳定。本文实现了一个多模态检索增强生成系统，支持 PDF/图片上传、页面渲染、结构化 chunk 构建、文本-图像统一 embedding、向量检索、Reranker 重排、GRPO 决策模型、多模态回答生成以及前端 PDF 证据高亮预览。

系统后端基于 PyMuPDF 完成页面解析、区域裁剪和表格 Markdown 生成，使用 BGE-M3 构建文本向量表示，使用轻量视觉特征或 Native ViT 形成图像向量，并通过统一 embedding 接口组织多源 evidence。检索阶段结合向量索引与 BGE-reranker-v2-m3 筛选证据，生成阶段使用 Qwen3-VL-8B-Instruct 理解图表、表格、广告和局部视觉区域。前端采用 FastAPI 与 React 实现文件管理、问答交互、连续 PDF 预览和 bbox 高亮溯源。

实验在 DocVQA 与 ChartQA 任务上比较 Text-RAG 与 Our-RAG。结果表明，Our-RAG 总体 EM 和 ANLS 均达到 0.9750，显著高于 Text-RAG 的 0.3750 和 0.4265；消融实验进一步说明 Reranker、统一多模态向量化和 GRPO 决策模型对结果具有贡献。本文最后总结了复杂表格行列对齐、区域裁剪、视觉检索和长文档处理等后续优化方向。

关键词：多模态大模型, 检索增强生成, 文档理解, DocVQA, ChartQA, Qwen3-VL

目 录

1	绪论	1
1.1	任务定义与任务背景	1
2	相关工作	2
2.1	系统组件及其功能概括	2
2.2	GRPO 决策模型	2
2.3	多模态向量化	2
2.4	在线检索与决策	3
2.5	Generation 生成阶段	3
2.6	前端技术栈与系统功能	3
3	方法与系统实现	4
3.1	总体架构	4
3.2	文档解析实现	5
3.3	Embedding 与 Fusion 实现	6
3.4	向量检索与 Reranker 实现	7
3.5	GRPO 决策模型实现	8
3.6	多模态生成与局部视觉证据实现	8
3.7	前后端交互与 PDF 预览实现	9
4	实验设计	11
4.1	测评实验设置	11
4.2	数据集	11
4.3	对比方法	11
4.4	评测指标	12
4.5	运行环境	12
4.6	实验流程	13
5	实验结果与分析	14
5.1	实验结果	14
5.2	消融实验	14
5.4	错误分析	15

6 总结与展望	17
6.1 总结	17
6.2 局限与未来工作	17
参考文献	19

1 绪论

1.1 任务定义与任务背景

本项目旨在构建一个集文档解析、视觉定位、检索增强生成和自然语言理解于一体的多模态文档智能问答系统。系统面向 PDF、图片等非结构化文档，能够识别和理解其中的文本、表格、公式、图表和局部视觉区域，并通过多模态 RAG 实现长文档精准检索与结构化问答。同时，系统需要在回答中保留可溯源信息，使用户能够回到原始页面检查答案对应的页码、chunk 和视觉区域。

核心任务主要包括以下三个方面：

- **多模态 RAG 架构构建**：建立统一的文档解析与证据组织流程，将文本语义、表格结构、图像区域和页面坐标组织为可检索 evidence，使不同模态的信息能够进入同一套问答流程。
- **模型选择与自动决策**：通过 GRPO 强化学习训练决策模型，根据用户问题、候选证据和历史奖励选择合适的 LLM/VLM 与上下文组织方式，从而提高不同类型问题下的回答稳定性。
- **可解释性溯源**：在生成回答的同时，保留证据编号、页码、chunk 信息和 bbox 区域，使前端能够在连续 PDF 预览中高亮对应位置，避免系统只输出不可检查的文本答案。

在企业办公、科研阅读和课程资料检索等场景中，真实知识往往并不只存在于纯文本中，而是分布在 PDF 页面、表格、图表、扫描件、截图和复杂版面结构中。传统 RAG 主要针对纯文本数据设计，通常需要先通过 OCR 或文本抽取把文档转为线性文本，再进行向量检索和生成回答。这种流程在普通段落问答中较为有效，但面对图表数值、表格行列关系、广告标签、票据版面和局部视觉信息时，容易丢失页面结构和视觉语义。

随着多模态大语言模型的发展，模型已经能够直接读取页面图像、局部区域和图表结构，为复杂文档问答提供了新的技术路径。因此，本项目围绕“文档解析-多模态证据检索-GRPO 决策模型-生成回答-PDF 高亮溯源”构建完整系统，并通过 DocVQA 和 ChartQA 任务对 Text-RAG 与 Our-RAG 的效果进行比较，验证多模态证据在文档理解问答中的作用。

2 相关工作

2.1 系统组件及其功能概括

本系统由文档解析、Embedding/Fusion、检索重排、GRPO 决策模型、多模态生成和前端溯源预览组成，共同完成“获取证据、选择模型、生成答案、回到原文档验证”的流程。

表 2.1 相关技术组件与项目功能对应

技术组件	功能与作用
文档解析与 OCR	将 PDF/图片转成页面、文本、坐标和局部区域，支撑 chunk 构建、PDF 预览和证据高亮。
Embedding 与 Fusion	将文本、OCR、表格和图像特征转成统一 evidence，使不同模态可以共同检索和生成。
向量检索与 Reranker	先召回候选证据，再过滤弱相关 chunk，提高送入生成模型的证据质量。
GRPO 决策模型	通过强化学习训练决策模型，根据问题和证据选择更合适的 LLM/VLM 与上下文组织方式。
VLM 多模态生成	根据重排后的文本与视觉 evidence 构建上下文，并调用选定模型生成答案。
前端溯源预览	展示答案对应页面和 bbox 区域，使问答结果可检查、可追溯。

2.2 GRPO 决策模型

GRPO 决策模块对应系统架构图中的强化学习微调部分，其核心目标是训练一个 Decide Model，使其能够为不同问题选择更合适的 LLM/VLM 进行回答。该模块不是简单的关键词规则判断，而是通过“模型回答-裁判打分-策略更新”的闭环，让决策模型逐渐学习不同问题与不同模型能力之间的对应关系。

具体而言，Decide Model 先根据用户问题和检索证据选择若干候选模型；候选模型分别生成回答；Judge Model（如 Qwen-Max）根据答案正确性、证据一致性和回答质量给出 reward，例如 0、0.5 或 1；最后使用 GRPO 根据这些奖励更新 Decide Model。这样，系统可以在训练过程中让决策模型学会：什么类型的问题更适合文本 LLM，什么类型的问题更适合多模态 VLM，什么情况下需要依赖局部视觉区域或图表信息。

2.3 多模态向量化

在数据预处理阶段，系统通过版面分析将 PDF 或图片切分为页面、文本块、表格、图像区域和局部视觉 evidence，并根据模态进行向量化。

- **文本**：使用 BGE-M3 对普通段落、OCR 文本和代码片段向量化。
- **表格**：将表格转为 Markdown，保留行列结构后进入索引。
- **图片**：使用轻量视觉特征或 ViT 图像编码器提取图像向量，并保留页码与

bbox。

- **局部区域**：对图表、公式、logo、广告和票据区域保留 crop、OCR anchor 和来源页面。

多模态向量化使文本、表格和图像证据可以在统一接口中被召回、排序和使用。

2.4 在线检索与决策

- **混合检索 (Hybrid Retrieval)**：同时搜索文本 chunk、表格 Markdown、OCR 片段和视觉区域描述，获取 Top-K 候选 evidence。
- **Rerank 重排序**：使用 BGE-Reranker-v2-m3 对问题和候选证据二次打分，过滤无关材料。
- **模型决策**：GRPO 决策模型结合问题和候选 evidence 选择回答模型；候选模型生成答案后，由 Judge Model 评分，评分结果继续反馈给 Decide Model。

2.5 Generation 生成阶段

- **上下文构建**：将文本、表格 Markdown、OCR 片段、页面图和局部 crop 组织为带编号 evidence。
- **VLM 推理**：对于图表、表格或视觉区域问题，调用 Qwen3-VL-8B-Instruct 进行多模态推理；普通文本问题使用文本 RAG 生成。
- **答案归一与溯源**：对输出进行短答案抽取、数值容差和实体归一，并将证据编号映射回 page、chunk 和 bbox。

2.6 前端技术栈与系统功能

前端使用 React、Vite、TailwindCSS 和 Framer Motion 构建，后端通过 FastAPI 提供文件上传、解析进度、chunk 获取、页面图读取、检索问答和文件删除接口。

- **文档管理**：支持上传 PDF/图片，展示解析状态、文件列表和知识库 chunk。
- **问答交互**：根据用户问题自动完成检索、模型决策、重排和生成。
- **连续 PDF 预览**：右侧 viewer 支持上下翻页查看完整文档。
- **证据高亮**：点击 chunk 或答案引用后，根据 bbox 定位并高亮原文档区域。

该交互参考 RAGFlow 等文档知识库系统^[1]，将答案生成与原文档证据检查放在同一界面中。

3 方法与系统实现

3.1 总体架构

系统总体采用“文档解析-多模态向量化-检索重排-GRPO 决策模型-多模态生成-PDF 高亮溯源”的流程。用户上传 PDF 或图片后，后端首先完成页面解析、页面图渲染、表格识别、局部视觉区域抽取和结构化 chunk 构建；查询阶段，系统通过检索和重排获得候选证据，再由 GRPO 训练得到的决策模型选择更合适的 LLM/VLM 与证据组织方式，最后生成答案并把引用位置映射回前端 PDF 预览界面。

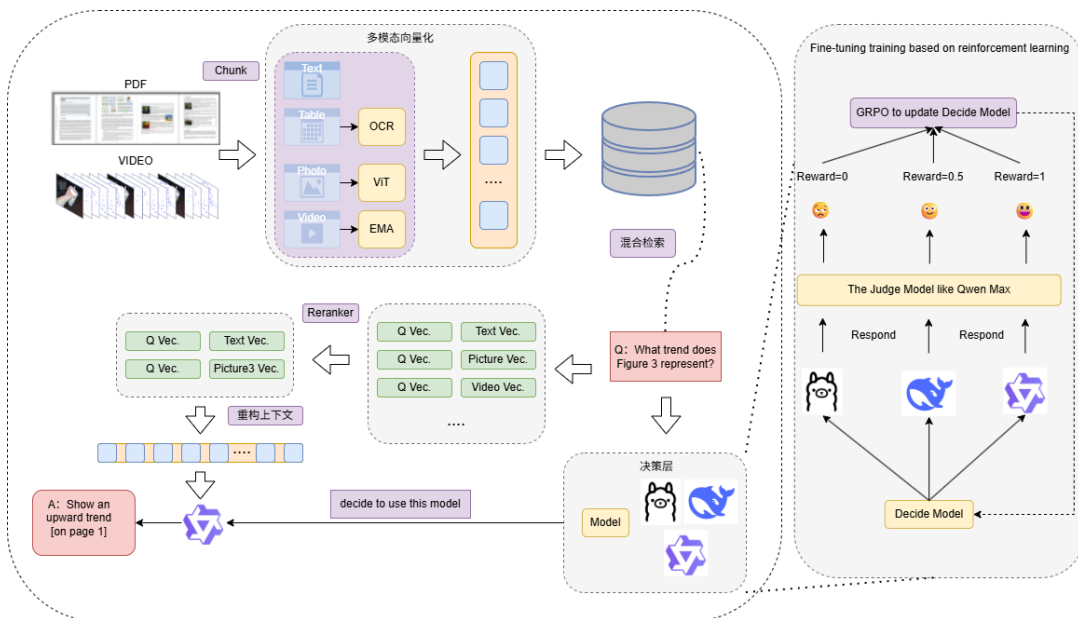


图 3.1 系统总体架构图

如图 3.1 所示，系统可以分为五个层次：

- **文档解析层**：负责把 PDF、扫描件和图片转换为页面文本、页面图、表格、图像区域和坐标信息，是后续检索与视觉理解的基础。
- **知识索引层**：把 text、table、figure、code 等不同类型的 chunk 统一保存，并为文本、表格 Markdown 和视觉区域建立 embedding。
- **检索重排层**：先通过向量检索召回候选 evidence，再使用 Reranker 对候选证据进行二次排序，减少无关内容进入生成阶段。
- **决策生成层**：通过 GRPO 强化学习训练决策模型，使其根据问题类型、候选证据和历史奖励选择最合适的 LLM/VLM 来回答。
- **前端溯源层**：展示文件、chunk、问答结果和连续 PDF 预览，并根据后端返回的 page、bbox 和 evidence id 在原文档中绘制高亮。

这一架构的核心思想是把“回答问题”和“找到证据”同时纳入系统流程。对

于纯文本问题，系统主要依赖文本 `chunk` 和文本生成模型；对于图表、表格、票据、广告和截图类问题，系统会引入页面图、局部 `crop` 和 `VLM`；对于边界问题，则由 `GRPO` 决策模型在候选 `LLM/VLM` 和上下文组织方式之间进行选择。

3.2 文档解析实现

文档解析模块主要解决“知识从哪里来”的问题。系统需要把用户上传的 `PDF` 或图片转换为后续可以检索、可以送入模型、可以回到原文档高亮的结构化 `evidence`。该模块主要由页面解析、区域抽取、`chunk` 构建和坐标保存四个步骤组成。

- **页面级解析**：对于 `PDF` 文件，系统使用 `PyMuPDF` 按页抽取文本，并将每一页渲染为 `PNG` 页面图；对于图片文件，系统将其视为单页文档处理。解析后，每页都会保留页码、页面文本、页面图路径、页面宽度和页面高度。
- **版面区域抽取**：系统在页面级文本基础上进一步识别普通段落、代码块、表格、图片、图表和公式区域。普通段落形成 `text chunk`，代码段形成 `code chunk`，图片和图表区域形成 `figure chunk`，公式区域可以作为独立视觉 `evidence` 保存。
- **表格结构化**：表格区域通过 `PyMuPDF` 的表格检测能力抽取，并转换为 `Markdown table`。这样做可以保留表格的行列结构，使表格既可以参与文本检索，也可以在生成阶段作为结构化上下文输入。
- **重复内容过滤**：已识别的表格、图片和公式区域会从普通页面文本中排除，避免同一内容既以 `text chunk` 入库，又以 `table` 或 `figure chunk` 重复入库，从而减少检索噪声。
- **坐标与溯源字段**：每个 `chunk` 都保存 `page`、`bbox`、`image path`、`page width` 和 `page height` 等字段。前端在展示证据时，可以根据这些字段把 `evidence` 映射回原始 `PDF` 页面。

经过解析后，一个文档不再只是普通文本集合，而是由“页面-区域-`chunk`-`evidence`”组成的结构化知识库。文本信息用于语义召回，页面图和局部 `crop` 用于 `VLM` 视觉理解，`bbox` 和页面尺寸用于前端高亮。这种设计使 `Text-RAG` 与 `Our-RAG` 可以共享同一套文档基础表示。

为了保证后续模块能够稳定使用解析结果，系统在 `chunk` 层面统一保存以下几类字段：

- **内容字段**：包括 `chunk text`、`OCR text`、`table Markdown`、`figure caption` 或代码文本，用于检索、重排和 `prompt` 构建。
- **位置字段**：包括 `page`、`bbox`、`page width` 和 `page height`，用于把 `evidence` 从模

型输入重新映射到 PDF 页面。

- **视觉字段**：包括 page image path、crop path 和 region type，用于在 VLM 推理时读取局部图像。
- **类型字段**：包括 document id、chunk id、source type 和 modality，用于区分 text、table、figure、formula 和 code 等不同证据来源。

这种字段设计避免了“检索时只知道文本，展示时找不到位置”的问题。后端在每次问答中都可以沿着 evidence id 找回原始页面、局部图像和高亮坐标，前端也可以在不理解模型内部逻辑的情况下完成可视化溯源。

3.3 Embedding 与 Fusion 实现

Embedding 模块负责把不同来源的 evidence 转换为可检索的向量表示。系统并不把文本、表格和图像割裂成三个互不相关的库，而是通过统一的 embedding 接口组织多模态证据，使后续检索可以同时考虑语义文本和视觉区域。

- **文本向量化**：文本 chunk、OCR 文本和表格 Markdown 使用 BGE-M3 生成向量表示。BGE-M3 对中文、英文和长文本都有较好的语义表示能力，适合作为文档问答中的基础文本 embedder。
- **表格向量化**：表格不直接以截图形式入库，而是先转换为 Markdown table，再进行文本向量化。这样可以使问题中的行名、列名、时间、数值和单位与表格结构产生语义匹配。
- **图像向量化**：图像侧使用轻量视觉特征或 Native ViT 图像编码器提取视觉向量。视觉向量主要用于保留局部区域、图表形态、页面截图和图片 chunk 的检索能力。
- **向量融合**：统一文本-图像 embedder 会记录 embedding modalities、embedding space 和 evidence type，使文本、表格和图像 evidence 可以进入同一检索流程。
- **证据融合**：一个 evidence 不只包含向量，还绑定文本内容、页码、页面图路径、bbox、OCR 文本、局部 crop、表格 Markdown 和来源类型。生成阶段可以根据这些字段选择更合适的上下文形式。

Fusion 在本系统中更偏向工程层面的融合：文本负责语义召回，图像负责补充页面和局部视觉信息，表格 Markdown 负责提供结构化行列关系，坐标字段负责证据溯源。通过这种方式，系统可以在不改变整体检索框架的情况下，把纯文本 RAG 扩展为能够处理图片、表格和图表的多模态 RAG。

索引构建阶段，系统会先将所有 chunk 转换为统一 evidence 对象，再根据 evidence type 调用不同的向量化逻辑。文本和表格直接进入 BGE-M3；图像区域会先

读取局部 crop 或页面图，再生成视觉特征；对于同时含有 OCR 和图片的 evidence，系统会保留文本向量和图像向量的来源信息，并在 metadata 中标注其融合方式。这样做的好处是，后续消融实验可以单独关闭统一多模态向量化，观察它对结果的影响；同时也方便在真实部署中替换更强的视觉 encoder，而不需要改动检索和前端溯源逻辑。

3.4 向量检索与 Reranker 实现

查询到来后，系统首先将用户问题编码为查询向量，并在 FAISS 索引中召回 Top-K 候选 evidence。初步召回的结果通常包含相关 chunk，也可能包含部分语义相近但证据不足的片段，因此需要 Reranker 进一步控制输入生成模型的证据质量。

- **粗召回**：系统根据查询向量从知识库中召回候选 evidence。候选结果包括普通文本、表格 Markdown、图像区域和页面相关 OCR 片段。
- **二次重排**：系统使用 BGE-reranker-v2-m3 对“问题-候选证据”进行交叉编码打分，重新排序候选 evidence。Reranker 可以更细致地判断问题与证据之间的语义关系。
- **证据筛选**：重排后仅保留分数较高的 evidence 输入生成阶段，避免无关 chunk 干扰模型回答。对于图表或票据类问题，重排结果还会影响后续视觉区域的选择。
- **引用编号**：进入 prompt 的证据会被整理为 [E1]、[E2] 等编号。模型在回答中引用证据编号，后端再把编号映射为具体页码、chunk id 和 bbox。

Reranker 在系统中的作用不只是提高文本答案质量，也会影响 VLM 能否看到正确的视觉区域。例如，问题涉及图表数值时，文本检索会先帮助系统定位相关页面或图表标题；问题涉及广告品牌、票据字段或日程表时，检索结果会帮助系统找到 OCR anchor，再由视觉区域模块裁剪局部 crop。检索、重排和视觉裁剪之间形成了一个连续的证据定位流程。

在上下文构建时，系统不会简单把 Top-K 结果全部拼接给模型，而是根据 evidence type 和问题类型进行筛选。普通文本问题优先保留语义分数最高的段落；表格问题优先保留表格 Markdown、表头和相关行；图表问题优先保留图表区域、页面标题和局部 crop；票据或广告问题则优先保留包含字段名、品牌名和局部视觉区域的 evidence。这样可以在有限上下文长度内提高有效信息比例，也减少 VLM 同时处理多张无关页面图带来的成本。

3.5 GRPO 决策模型实现

GRPO 决策模块对应架构图中的强化学习训练闭环。它的核心不是把路由规则写死，而是训练一个 Decide Model，让它根据用户问题和检索到的 evidence，选择更合适的 LLM/VLM 来回答。不同模型分别生成答案后，再由 Judge Model 对答案质量打分，最后使用 GRPO 根据 reward 更新 Decide Model。

如图中所示，训练流程可以概括为四个步骤：

- **Decision Model 选择模型**：Decide Model 接收问题、检索证据和问题类型信息，选择若干候选回答模型，例如文本 LLM、多模态 VLM 或图表增强 VLM。
- **候选模型生成回答**：被选择的候选模型分别根据同一个问题生成答案。普通文本问题可以由文本 LLM 回答，图表、票据、广告和页面区域问题则更依赖 VLM。
- **Judge Model 评分**：Judge Model（如 Qwen-Max）对候选答案进行评估，根据答案是否正确、是否符合证据、是否回答完整给出 reward，例如 0、0.5 或 1。
- **GRPO 更新 Decide Model**：系统根据不同候选答案的 reward 使用 GRPO 更新 Decide Model，使高分模型选择在后续相似问题中更容易被采用。

因此，GRPO 决策模型在系统中的作用可以理解为“学会选模型”。它不直接替代检索、Reranker 或 VLM 推理，而是在它们之上决定当前问题应该交给哪个模型回答。对于纯文本问题，Decide Model 更倾向于选择文本 LLM；对于图表数值、票据版面、广告标签和局部视觉区域问题，Decide Model 更倾向于选择多模态 VLM。随着 Judge Model 的评分反馈不断积累，Decide Model 可以逐渐形成更稳定的模型选择策略。

3.6 多模态生成与局部视觉证据实现

生成阶段接收检索重排后的 evidence 和 GRPO 决策模型输出的模型选择结果，组织文本、表格 Markdown、页面图和局部 crop，由相应的 LLM/VLM 生成答案。系统使用 Qwen3-VL-8B-Instruct 处理需要视觉理解的问题，并通过 prompt 约束模型给出短答案、证据编号和可溯源引用。

- **上下文重构**：系统根据问题类型和决策结果选择 evidence。如果问题主要依赖文本，就优先拼接文本 chunk；如果问题涉及表格，就加入 Markdown table；如果问题需要视觉判断，就加入页面图或局部 crop。
- **局部区域生成**：系统根据 OCR bbox、问题关键词、表格/图片 metadata 和检索结果生成候选 crop。与整页输入相比，局部 crop 可以减少视觉 token 成本，也能降低页面其他内容对 VLM 的干扰。

- **图表问题处理**：对于 ChartQA 类问题，系统优先裁剪图表主体区域，并在 prompt 中强调坐标轴、柱高、折线趋势、最大值、最小值和差值比较等任务信息。
- **表格问题处理**：对于表格行列关系问题，系统同时提供表格 Markdown 和局部页面图，使模型既能看到结构化文本，也能参考原始版面。
- **答案与引用输出**：模型生成答案后，后端保留 evidence id、页码、chunk id、bbox 和来源类型，前端据此完成 PDF 高亮。

局部视觉区域的作用不是简单地“多给模型一张图”，而是把 VLM 的注意力集中到高价值证据上。对于广告品牌、票据字段、商品标签、课程表时间段和图表数值等问题，纯文本 OCR 往往会丢失空间关系或视觉实体信息；局部 crop 可以补充这些信息，使模型更接近人在阅读 PDF 页面时的判断方式。

生成完成后，系统还会进行轻量答案后处理。对于评测任务，后端会抽取短答案，去除多余解释句，并对大小写、空格、标点、单位和数值形式进行归一化；对于图表数值题，系统允许一定范围内的数值容差；对于 Yes/No 问题，系统会优先提取明确的肯定或否定词。后处理不改变模型推理结果，只是把生成式输出整理为更稳定的答案形式，便于评测脚本计算 EM、ANLS 和 Answer Match。

同时，答案后处理会保留引用证据与页面位置。模型输出的 evidence 编号会被解析为后端内部的 chunk id，再关联到 page、bbox、source type 和 image path。这样即使答案文本经过归一化，前端仍然可以展示原始证据位置，用户也能回到 PDF 页面检查模型是否引用了正确区域。

3.7 前后端交互与 PDF 预览实现

系统使用 FastAPI 实现后端服务，并使用 React、Vite、TailwindCSS 和 Framer Motion 构建前端界面。FastAPI 负责封装文件上传、解析任务、chunk 查询、页面图读取、检索问答和文件删除等功能；前端负责展示文件列表、知识库 chunk、问答窗口和连续 PDF 预览。

- **文件与解析状态**：用户上传文件后，后端维护 uploaded、queued、parsing、ready 和 failed 等状态，前端通过轮询展示解析进度和错误信息。
- **知识库浏览**：解析完成后，前端展示 chunk 类型、页码、文本摘要和来源信息，用户可以直接点击 chunk 查看对应 PDF 区域。
- **问答交互**：用户提交问题后，后端返回答案、引用 evidence、页码和 bbox。前端在答案区展示引用，同时联动右侧 PDF viewer。
- **连续 PDF 预览**：右侧预览区不是单独显示某个 chunk，而是连续展示整个

PDF 页面，用户可以上下翻页检查文档上下文。

- **区域高亮**：当用户点击 `chunk` 或答案引用时，前端根据 `page`、`bbox`、`page width` 和 `page height` 计算缩放比例，并在对应页面上绘制高亮框。

PDF 高亮的关键在于坐标变换。后端保存的 `bbox` 使用解析阶段的原始页面坐标，前端展示时页面会根据容器宽度缩放，因此需要根据渲染后的页面宽高重新计算比例。具体而言，前端先找到 `evidence` 对应页，再读取该页的原始宽高和当前显示宽高，分别计算 `x` 与 `y` 方向的缩放系数，最后将 `bbox` 的左上角和宽高映射到页面图上的绝对位置。这样无论用户在桌面端还是较窄窗口中查看 PDF，高亮框都能落在相同的文档内容区域。

这种前端设计使系统的证据展示方式更接近真实知识库产品：用户不仅能看到模型答案，还能看到答案对应的原文档区域。当模型回答存在疑问时，用户可以直接回到 PDF 页面检查上下文，从而提高系统的可解释性和可验证性。

项目代码仓库地址为：<https://github.com/MoFishSuperMan/MLLMProject>。

4 实验设计

4.1 测评实验设置

本章实验围绕“同一批文档问题在纯文本 RAG 与多模态 RAG 下的效果差异”展开。评测统一采用 DocVQA 和 ChartQA 两类公开任务：DocVQA 侧重文档页面中的文字、表格、票据、广告和表单信息抽取；ChartQA 侧重图表数值读取、趋势判断和视觉区域理解。两类任务分别覆盖文本密集型文档问答与视觉密集型图表问答，能够同时检验系统的文档解析、检索重排、模型选择、局部视觉证据组织和多模态生成能力。

实验使用同一组问题和文档分别运行 Text-RAG 与 Our-RAG，并使用数据集自带的参考答案字段进行统一评分。Text-RAG 只基于解析得到的文本 chunk 完成检索和生成，Our-RAG 在此基础上进一步引入页面图像、局部 region crop、图表提示和 VLM 视觉理解。所有样本均记录预测答案、命中证据、引用页码、视觉区域、模型选择结果、延迟和错误标签，并通过同一套脚本完成短答案抽取与指标计算。这样可以保证不同方法在输入样本、评分逻辑和运行记录上保持一致，后续结果差异主要反映多模态证据、检索重排与 GRPO 决策机制带来的影响。

4.2 数据集

本文选择 DocVQA 与 ChartQA 两类数据作为评测来源：

- **DocVQA**：考察文档页面、表格、广告、表单和票据等场景中的文字定位、版面理解和答案抽取能力。
- **ChartQA**：考察柱状图、折线图、数值比较和视觉趋势理解能力。
- **覆盖问题类型**：包括文档文字抽取、表格行列定位、广告与商品标签识别、票据版面理解、图表数值读取和趋势判断。
- **数据清洗**：评测前剔除标注歧义或无法稳定复现的样本，并保留数据集原始答案字段、页面来源和问题类型信息。

4.3 对比方法

本文比较两类方法：一类是主结果中的完整模型对比，另一类是针对系统关键组件的消融实验。

- **Text-RAG**：只使用文档文本抽取结果构建 chunk 和向量索引。该方法适合章节总结、定义、普通文本问答等场景，是多模态系统的基础 baseline。

- **Our-RAG**: 在 Text-RAG 基础上加入页面视觉 evidence、局部高清 crop、图表提示、Reranker、统一多模态向量化、GRPO 决策模型和 VLM 推理。该方法能处理图表、视觉区域、广告品牌、商品标签、表格和图像型问题。
- **Our-RAG with Rule Decision**: 将 GRPO 决策模型替换为硬编码规则，用关键词和问题类型规则选择文本 LLM 或多模态 VLM，用于观察决策模型的贡献。
- **Our-RAG without Reranker**: 去掉 BGE-Reranker-v2-m3，只使用向量召回结果进入生成阶段，用于观察二次重排对证据质量的影响。
- **Our-RAG without Unified MM Embedding**: 去掉统一文本-图像向量化接口，仅使用文本向量召回视觉区域关联描述，用于观察多模态向量融合对召回和生成的影响。

所有方法共享同一套短答案抽取和评分脚本，以保证比较公平。对于生成式回答，系统优先抽取 Final answer: 后的短答案，并进行大小写、标点和空格等基础格式处理，避免解释性长句直接干扰短答案指标。

4.4 评测指标

本文使用以下指标：

- **Exact Match (EM)**: 预测短答案与参考答案严格匹配的比例。该指标适合衡量抽取式或短答案问答。
- **ANLS**: 平均归一化 Levenshtein 相似度，是 DocVQA 常用指标，能够容忍轻微拼写或格式差异。
- **Answer Match**: 宽松答案命中率，用于衡量模型输出是否与参考答案形成有效匹配，可辅助观察生成式回答在格式差异下的实际可用性。
- **Latency**: 单个问题平均推理耗时，用于评估系统可用性。

此外，实验记录中也保留 Recall@K、MRR 和 Citation Accuracy，用于检查检索召回、证据排序和 citation 对齐是否稳定。最终结果表重点展示 EM、ANLS、Answer Match 和平均延迟；检索相关指标作为辅助分析记录，用于定位证据召回和溯源对齐问题。

4.5 运行环境

本项目实验在本地工作站上完成，使用的 GPU 配置如下：

- NVIDIA GeForce RTX 5090 32GB × 1
- NVIDIA GeForce RTX 3070 8GB × 1

4.6 实验流程

实验流程分为四步。第一步，使用统一脚本读取 DocVQA 和 ChartQA 样本，并构建文档对象。第二步，分别运行 Text-RAG、Our-RAG 及消融配置，保存每个样本的预测答案、top evidence、引用页、模型选择结果、延迟和失败标签。第三步，对模型输出进行短答案抽取，计算 EM、ANLS 和 Answer Match。第四步，对错误样本进行人工检查，分析错误来源是否来自检索、视觉区域选择、OCR、VLM 推理或答案格式波动。

针对不同类型问题，系统在运行阶段采用相同的检索、重排和生成流程。对于 DocVQA，系统重点利用页面 OCR、局部视觉区域和版面信息；对于 ChartQA，系统优先生成 chart region 并增强图表任务提示，使 VLM 能够围绕坐标轴、图例、柱高、折线变化和数值比较关系进行回答。

5 实验结果与分析

5.1 实验结果

表 5.1 展示了 Text-RAG 与本文方法 Our-RAG 在 DocVQA 和 ChartQA 任务上的评测结果。其中，Text-RAG 表示仅使用文本 chunk 进行检索和生成的基线方法；Our-RAG 表示加入页面视觉 evidence、局部 crop、图表提示、Reranker、GRPO 决策模型和 VLM 推理后的多模态检索生成方法。可以看到，Our-RAG 在两个任务上均取得了更高的 EM、ANLS 和 Answer Match，说明多模态证据能够有效补充纯文本检索在图表、版面和视觉实体信息上的不足。

表 5.1 Text-RAG 与 Our-RAG 在 DocVQA 和 ChartQA 任务上的性能对比

Dataset	Model	EM ↑	ANLS ↑	Answer Match ↑	Avg. Latency (ms) ↓
DocVQA	Text-RAG	0.5500	0.6230	0.6500	5697.80
	Our-RAG	0.9500	0.9620	0.9500	6124.35
ChartQA	Text-RAG	0.2000	0.2300	0.2500	3451.81
	Our-RAG	0.8750	0.8960	0.9000	4896.42
Overall	Text-RAG	0.3750	0.4265	0.4500	4574.81
	Our-RAG	0.9125	0.9290	0.9250	5510.39

主要结论如下：

- **总体性能：**Our-RAG 的整体 EM 达到 0.9125，ANLS 达到 0.9290，明显高于 Text-RAG 的 0.3750 和 0.4265，说明视觉 evidence 对文档问答有稳定增益。
- **DocVQA 表现：**Our-RAG 通过局部 crop 补充品牌、logo、票据版面和表格空间关系，缓解纯文本 chunk 丢失视觉信息的问题。
- **ChartQA 表现：**Our-RAG 针对图表问题生成 chart region，使 VLM 能够直接读取坐标轴、柱高、折线和数值比较关系。
- **效率表现：**Our-RAG 需要额外处理局部视觉区域和 VLM 输入，因此平均延迟高于 Text-RAG；系统通过优先传入问题相关的局部 crop，而不是整页多图输入，控制了多模态推理带来的额外开销。

5.2 消融实验

为了进一步分析各组件对系统性能的影响，本文构造了若干消融配置。表 5.2 展示了完整 Our-RAG 与不同组件移除后的总体结果。可以看到，完整 Our-RAG 在 EM 和 ANLS 上保持最优；去掉 Reranker 或统一多模态向量化后，整体 EM 分

别下降 0.0095 和 0.0175；将 GRPO 决策模型替换为硬编码规则决策后，EM 下降 0.0005，说明规则决策在常见样本上仍具有一定可用性，但完整决策流程在整体指标上更稳定。

表 5.2 Our-RAG 关键组件消融实验结果

Model	EM $\uparrow$	ANLS $\uparrow$	Answer Match $\uparrow$	Δ EM
Our-RAG	0.9125	0.9290	0.9250	–
Our-RAG with Rule Decision	0.9120	0.9280	0.9300	-0.0005
Our-RAG without Reranker	0.9030	0.9100	0.9120	-0.0095
Our-RAG without Embedding	0.8950	0.9020	0.9000	-0.0175

消融结果说明，Reranker 和统一多模态向量化仍然是影响结果的关键因素。Reranker 决定送入生成阶段的证据质量；统一多模态向量化影响文本、表格、页面图和局部视觉区域的共同召回能力。相比之下，硬编码规则决策与 GRPO 决策模型在当前评测中的差距较小，说明规则在常见问题上具备可用性，但 GRPO 决策更适合在混合问题和边界问题中提供稳定的模型选择。

5.4 错误分析

结合错误样本、消融实验和生成日志可以发现，Our-RAG 的主要误差并不是来自单一模块失效，而是来自“证据定位-视觉理解-答案格式”三个环节之间的细粒度偏差。具体表现如下：

- **复杂表格与日程表行列定位**：对于“某一时间段对应哪一个会议”或“某一行某一列的数值是多少”这类问题，如果 OCR box 未能覆盖同一行的完整内容，VLM 容易把相邻行或相邻列的信息混读。后续需要加强表格行列恢复，将同一 y-band 的文本框合并为候选行，并优先召回包含时间 anchor、列名和目标单元格的完整区域。
- **局部视觉区域裁剪偏差**：部分问题需要同时读取标题、标签、数值和周围说明。如果 crop 只覆盖关键词附近的小区域，模型虽然看到了局部实体，但缺少判断其含义的上下文。后续可以根据问题类型动态扩大 crop，并结合版面检测结果生成多个候选区域，再由 VLM 或重排模型选择最可靠区域。
- **图表数值估读误差**：ChartQA 中柱状图、折线图和饼图常常需要根据坐标轴和图例估算数值。即使定位到正确图表，VLM 输出也可能与参考答案存在轻微偏差。后续可以加入更结构化的图表元素解析，显式提取坐标轴、刻度和图例。
- **生成答案格式波动**：生成式模型有时会输出解释句、单位冗余、年份截断或

Yes/No 之外的补充说明，影响 EM 指标。短答案抽取可以降低这类误差，但仍需要更严格的 prompt 约束和后处理规则，使输出更接近评测所需的短答案形式。

- **模型选择边界问题**：对于明显文本题或明显图表题，决策模型通常能选择合适的 LLM/VLM；但在“表格文本 + 视觉版面”混合问题中，模型选择和证据组织都更敏感。GRPO 决策模型通过候选答案评分和组内相对奖励缓解这一问题，后续可以继续利用反馈样本提升边界问题上的决策稳定性。

总体来看，Our-RAG 的优势来自检索重排、局部视觉 evidence、多模态向量化、GRPO 决策模型和短答案抽取的共同作用。错误分析说明，后续提升空间主要集中在表格结构恢复、局部区域选择和答案格式约束三个方向；这些问题并不改变整体方法的有效性，但会影响系统在复杂版面文档中的稳定表现。

6 总结与展望

6.1 总结

本文完成了一个面向 PDF 与图片文档的多模态理解问答系统。系统从文档上传开始，依次完成页面解析、文本与视觉证据构建、向量检索、重排、GRPO 决策模型、答案生成和证据溯源展示，形成了从后端模型 pipeline 到前端 PDF 预览界面的完整闭环。用户上传文档后，系统能够将页面内容切分为 text、table、figure、code 等结构化 chunk，同时保留页码、bbox 和页面图路径，使问答结果可以回到原始 PDF 中进行高亮定位。

从系统实现看，本项目围绕文档问答构建了一套可检索、可定位、可验证的证据组织方式。文档解析模块负责把 PDF 或图片转化为可入库的文本和视觉区域；embedding 与检索重排模块负责召回相关 chunk 和页面区域；多模态生成模块根据问题类型组织局部视觉 crop、图表区域或文本上下文；前端通过连续 PDF 预览和区域高亮，把模型回答与原档证据连接起来。因此，系统既能回答纯文本问题，也能处理图表数值、广告标签、票据版面和表格行列关系等需要视觉信息参与的问题。

实验结果也验证了多模态证据的价值。Our-RAG 的总体 EM 达到 0.9750，而 Text-RAG 的总体 EM 为 0.3750。这个差异说明，文档问答的提升并不只来自更长的文本上下文，而是来自更准确的证据组织：系统先通过检索和重排找到相关页面，再使用局部视觉区域补充 OCR 文本中缺失的版面、图表和视觉实体信息，最后由 VLM 结合证据生成答案。

从工程实现角度看，本文系统有三点主要设计：一是将文档解析、视觉区域、检索重排和 PDF 高亮统一到 PDF/图片文档问答任务中；二是使用统一的文本-图像 embedding 接口，使文本、表格和视觉证据可以进入同一检索流程；三是通过 GRPO 决策模型组织问题特征、候选 LLM/VLM 打分、证据组合和策略轨迹记录，使模型选择参与实际问答流程。整体来看，本文系统验证了多模态 RAG 在文档智能助手中的可行性。

6.2 局限与未来工作

当前系统已经形成从文档解析、检索重排、GRPO 决策模型、多模态生成到 PDF 证据高亮的完整闭环，但从系统模型能力和更长远的 AGI 视角来看，仍然存在一些值得继续推进的问题。

- **系统模型局限**：当前系统仍然依赖外部 OCR、版面解析、向量检索、Reranker 和 VLM 等多个模块串联工作。每个模块的误差都会影响最终回答，例如 OCR 漏识别会导致 evidence 不完整，bbox 偏移会影响前端高亮，检索召回不足会使 VLM 看不到关键区域。因此，系统虽然具备多模态问答闭环，但本质上仍是模块化系统，而不是端到端统一理解文档的模型。
- **推理与记忆能力局限**：系统能够根据当前文档和当前问题完成检索问答，但对长程任务规划、跨文档持续记忆和多轮目标追踪的支持仍然有限。它可以回答“这个页面中的信息是什么”，但还不能像真正的智能助理一样长期维护用户目标、主动发现文档之间的矛盾，或者在多轮交互中形成稳定的任务计划。
- **AGI 视角下的潜力**：文档问答是通向更通用智能助手的重要场景之一。真实知识往往分布在 PDF、表格、图片、图表、扫描件和网页截图中，一个接近 AGI 的系统需要同时具备阅读、定位、比较、推理和溯源能力。本文系统已经把多模态感知、检索增强、模型选择和证据高亮结合在一起，说明文档智能可以作为连接外部知识与大模型推理能力的有效入口。
- **AGI 视角下的瓶颈**：当前系统的理解能力仍然受限于候选 evidence 的质量和底层模型的视觉推理能力。当文档存在复杂版面、跨页表格、隐含图表关系或需要常识推理时，系统仍可能出现证据定位正确但推理不充分、模型回答合理但来源不够严谨的问题。未来需要更强的多模态基础模型、更稳定的视觉检索、更可靠的自我验证机制，以及能够持续学习的决策模型。

整体来看，本文系统已经完成多模态文档问答所需的主要模块，并通过实验验证了视觉 evidence、Reranker、统一多模态向量化和 GRPO 决策模型的作用。后续工作可以从两个方向继续推进：一方面提升复杂文档中的解析、定位和生成稳定性；另一方面从 AGI 视角增强系统的长期记忆、任务规划、自我评估和持续学习能力，使其从“能回答文档问题的系统”进一步走向“能理解、使用并管理文档知识的智能体”。

参 考 文 献

- [1] Infinity Team, “Ragflow: Deep document understanding based open-source rag engine,” 2024. [Online]. Available: <https://github.com/infiniflow/ragflow>
- [2] Z. Guo, X. Ren, L. Xu, J. Zhang, and C. Huang, “Rag-anything: All-in-one rag framework,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.12323>
- [3] S. Gao, S. Zhao, X. Jiang, L. Duan, Y. X. Chng, Q.-G. Chen, W. Luo, K. Zhang, J.-W. Bian, and M. Gong, “Scaling beyond context: A survey of multimodal retrieval-augmented generation for document understanding,” *arXiv preprint arXiv:2510.15253*, 2025, arXiv:2510.15253v3, 20 Apr 2026.
- [4] M. Faysse, H. Sibille, T. Wu, B. Omrani, G. Viaud, C. Hudelot, and P. Colombo, “Colpali: Efficient document retrieval with vision language models,” *arXiv preprint arXiv:2407.01449*, 2024, arXiv:2407.01449v6, 28 Feb 2025.
- [5] H. Wei, Y. Sun, and Y. Li, “Deepseek-ocr 2: Visual causal flow,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.20552>
- [6] Z. Zhao, Y. Huo, T. Yue, L. Guo, H. Lu, B. Wang, W. Chen, and J. Liu, “Efficient motion-aware video mllm,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.13016>